EXPERIMENT - 5

AIM

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

OBJECTIVES

- To create a view that summarizes employee salary information department-wise.
- To create an employee hierarchy view using manager relationships.
- To test whether a view is updatable and understand the conditions affecting view updates.
- To implement a recursive CTE for displaying employee reporting chains.
- To verify the results using SELECT queries.

DATABASE USED

The queries use the CompanyDB database and the Employee–Department–Project schema from the reference experiment. The Employee table is extended with manager_id to support employee hierarchy and recursive reporting chains.

```
USE CompanyDB;
```

SCHEMA REFERENCE

The Department, Project and Employee tables remain connected through dept_id and project_id. The Employee table additionally contains manager_id, which references Employee(emp_id).

SOURCE CODE

SQL QUERIES 1. DEPARTMENT SALARY SUMMARY VIEW

Create a view showing employee count, average salary, minimum salary, maximum salary and total salary for each department.

```
CREATE OR REPLACE VIEW department_salary_summary AS
SELECT
    d.dept_id,
    d.dept_name,
    COUNT(e.emp_id) AS employee_count,
    AVG(e.salary) AS average_salary,
    MIN(e.salary) AS minimum_salary,
    MAX(e.salary) AS maximum_salary,
    SUM(e.salary) AS total_salary
FROM Department d
```

```
JOIN Employee e
    ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Output

I/O | AI Agent | 101 ms

STDIN

Input for the program (Optional)

Output:

dept_id	dept_name	employee_count	average_salary	minimum_salary	maximum_salary	total_salary
1	Development	6	67833.333333	58000.00	81000.00	407000.00
2	PeopleOps	6	53666.666667	47000.00	61000.00	322000.00
3	Accounts	6	69166.666667	56000.00	83000.00	415000.00
4	Sales	6	60166.666667	51000.00	70000.00	361000.00
5	Logistics	6	66500.000000	59000.00	76000.00	399000.00

2. EMPLOYEE HIERARCHY VIEW

Create a view containing each employee and the name of the employee's immediate manager.

```
CREATE OR REPLACE VIEW employee_hierarchy AS
SELECT
    e.emp_id,
    e.emp_name,
    e.dept_id,
    d.dept_name,
    e.manager_id,
    m.emp_name AS manager_name,
    e.salary
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
LEFT JOIN Employee m
    ON e.manager_id = m.emp_id;
```

Output

I/O | AI Agent | 84 ms

STDIN
Input for the program (Optional)

Output:

emp_id	emp_name	dept_id	dept_name	manager_id	manager_name	salary
1	Ishaan	1	Development	NULL	NULL	65000.00
2	Reyansh	1	Development	1	Ishaan	72000.00
3	Adit	1	Development	2	Reyansh	58000.00
4	Aarush	1	Development	3	Adit	81000.00
5	Raghav	1	Development	4	Aarush	69000.00
6	Vihaan	1	Development	5	Raghav	62000.00
7	Anika	2	PeopleOps	NULL	NULL	52000.00
8	Kavya	2	PeopleOps	7	Anika	48000.00
9	Ira	2	PeopleOps	8	Kavya	61000.00
10	Naina	2	PeopleOps	9	Ira	55000.00
11	Aditi	2	PeopleOps	10	Naina	47000.00
12	Saanvi	2	PeopleOps	11	Aditi	59000.00
13	Harsh	3	Accounts	NULL	NULL	75000.00
14	Mohit	3	Accounts	13	Harsh	68000.00
15	Ritvik	3	Accounts	14	Mohit	83000.00
16	Ayush	3	Accounts	15	Ritvik	62000.00
17	Kunal	3	Accounts	16	Ayush	71000.00
18	Varun	3	Accounts	17	Kunal	56000.00
19	Mehak	4	Sales	NULL	NULL	54000.00
20	Pihu	4	Sales	19	Mehak	63000.00
21	Shreya	4	Sales	20	Pihu	57000.00
22	Tara	4	Sales	21	Shreya	66000.00
23	Mahi	4	Sales	22	Tara	51000.00
24	Anvi	4	Sales	23	Mahi	70000.00
25	Rajat	5	Logistics	NULL	NULL	60000.00
26	Yuvraj	5	Logistics	25	Rajat	73000.00
27	Arnav	5	Logistics	26	Yuvraj	67000.00
28	Laksh	5	Logistics	27	Arnav	59000.00
29	Devansh	5	Logistics	28	Laksh	76000.00
30	Om	5	Logistics	29	Devansh	64000.00

3. TEST UPDATABILITY OF VIEWS

Test an update on a simple employee hierarchy view and then test an update on the grouped department salary summary view.

```
UPDATE employee_hierarchy
SET salary = salary + 1000
WHERE emp_id = 6;
SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id = 6;
UPDATE department_salary_summary
SET average_salary = average_salary + 1000
WHERE dept_id = 1;
```

Output

The first UPDATE is expected to fail because the employee_hierarchy view contains joins and does not directly expose a single base-table row for every selected column. The grouped department_salary_summary view is also not directly updatable because it contains GROUP BY and aggregate functions. The base Employee table should be updated instead.

RECURSIVE CTE – REPORTING CHAIN

Display the reporting chain beginning with department heads and recursively follow manager relationships.

```
WITH RECURSIVE reporting_chain AS (
    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,          1 AS level,
        CAST(e.emp_name AS CHAR(500)) AS reporting_path
    FROM Employee e
```

```
    WHERE e.manager_id IS NULL
    UNION ALL
    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,         rc.level + 1,
        CONCAT(rc.reporting_path, ' -> ', e.emp_name)
    FROM Employee e
    JOIN reporting_chain rc
        ON e.manager_id = rc.emp_id
)
SELECT emp_id, emp_name, manager_id, level, reporting_path
FROM reporting_chain
ORDER BY reporting_path;
```

Output

I/O | AI Agent | 87 ms

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	manager_id	level	reporting_path
7	Anika	NULL	1	Anika
8	Kavya	7	2	Anika -> Kavya
9	Ira	8	3	Anika -> Kavya -> Ira
10	Naina	9	4	Anika -> Kavya -> Ira -> Naina
11	Aditi	10	5	Anika -> Kavya -> Ira -> Naina -> Aditi
12	Saanvi	11	6	Anika -> Kavya -> Ira -> Naina -> Aditi -> Saanvi
13	Harsh	NULL	1	Harsh
14	Mohit	13	2	Harsh -> Mohit
15	Ritvik	14	3	Harsh -> Mohit -> Ritvik
16	Ayush	15	4	Harsh -> Mohit -> Ritvik -> Ayush
17	Kunal	16	5	Harsh -> Mohit -> Ritvik -> Ayush -> Kunal
18	Varun	17	6	Harsh -> Mohit -> Ritvik -> Ayush -> Kunal -> Varun
1	Ishaan	NULL	1	Ishaan
2	Reyansh	1	2	Ishaan -> Reyansh
3	Adit	2	3	Ishaan -> Reyansh -> Adit
4	Aarush	3	4	Ishaan -> Reyansh -> Adit -> Aarush
5	Raghav	4	5	Ishaan -> Reyansh -> Adit -> Aarush -> Raghav
6	Vihaan	5	6	Ishaan -> Reyansh -> Adit -> Aarush -> Raghav -> Vihaan
19	Mehak	NULL	1	Mehak
20	Pihu	19	2	Mehak -> Pihu
21	Shreya	20	3	Mehak -> Pihu -> Shreya
22	Tara	21	4	Mehak -> Pihu -> Shreya -> Tara
23	Mahi	22	5	Mehak -> Pihu -> Shreya -> Tara -> Mahi
24	Anvi	23	6	Mehak -> Pihu -> Shreya -> Tara -> Mahi -> Anvi
25	Rajat	NULL	1	Rajat
26	Yuvraj	25	2	Rajat -> Yuvraj
27	Arnav	26	3	Rajat -> Yuvraj -> Arnav
28	Laksh	27	4	Rajat -> Yuvraj -> Arnav -> Laksh
29	Devansh	28	5	Rajat -> Yuvraj -> Arnav -> Laksh -> Devansh
30	Om	29	6	Rajat -> Yuvraj -> Arnav -> Laksh -> Devansh -> Om

RESULT

Views were created for department salary summaries and employee hierarchy. View update behavior was tested, and a recursive CTE was used successfully to display reporting chains.

CONCLUSION

SQL views provide reusable logical representations of data, while recursive CTEs can traverse selfreferencing employee relationships to display hierarchical reporting structures.